



Compiladores

Aula 4: Expressões regulares e autômatos finitos

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Alfabetos e linguagens formais
- Operações e expressões regulares
- Autômatos
- Aplicação no projeto MINIC

Alfabetos e linguagens formais

- Em **Teoria da Computação**, denominamos por **Alfabeto** um conjunto de símbolos utilizados para representar entidades maiores chamadas cadeias (strings).
- As **Linguagens Formais** correspondem às linguagens, i.e. conjuntos de strings, que obedecem certas regras de formação:
 - Regulares
 - Livres de contexto
 - Sensíveis ao contexto
 - Recursivamente enumeráveis

Alfabetos e linguagens formais

- Tomando todos os símbolos que podemos inserir em um código-fonte, obtemos um alfabeto, e os programas que escrevemos obedecem regras de formação que seguem os critérios de linguagens formais.
- Em particular, os lexemas que estudamos podem ser representados por linguagens regulares.

Alfabetos e linguagens formais

- Strings que pertencem a linguagens regulares podem ser produzidas por modelos denominados **expressões regulares** e reconhecidas por modelos denominados **autômatos finitos**.

01 Linguagens regulares

Alfabetos, cadeias, linguagens e operadores de expressões regulares.

02 Autômatos finitos

AFD, AFND e diagramas de transição como modelos de reconhecimento.

03 Aplicação ao MINIC

Tokens, prioridade de regras e preparação da Etapa 1.

Alfabetos e linguagens formais

- Como transformar uma especificação de token em um reconhecedor executável?

Alfabeto Σ

Conjunto finito de símbolos.

Ex.: $\Sigma = \{a, b, 0, 1\}$

Cadeia

Sequência finita de símbolos de Σ .

ϵ representa a cadeia vazia.

Linguagem

Conjunto de cadeias sobre um alfabeto.

Uma regra de token descreve uma linguagem.

Exemplo de linguagem

$L = \{ \text{cadeias que começam com uma letra e continuam com letras ou dígitos} \}$

Operações e expressões regulares

- Operadores:

- União: |
- Concatenação
- Estrela (Fecho): *

Precedência usual: * > concatenação > |

Use parênteses para deixar a intenção explícita.

- Através desses operadores, é possível escrever expressões regulares.
- Expressões regulares descrevem linguagens regulares de forma compacta, mesmo que haja muitas cadeias.

Operações e expressões regulares

- Exemplos:

$[1-9][0-9]^*$ → cadeias que começam com um dígito entre 1 e 9 seguido por dígitos entre 0 e 9.

$L = \{1, 2, 3, \dots, 10, 11, 12, \dots, 20, 21, 22, \dots\}$

$(a|e|i|o|u)[a-z]^*$ → cadeias que começam com uma vogal seguida por quaisquer letras entre a e z.

$L = \{a, aa, ab, ac, ad, \dots, e, ea, eb, ec, \dots, i, ia, ib, ic, \dots\}$

$(ab|bc|cd)$ → cadeias ab, bc ou cd.

$L = \{ab, bc, cd\}$

Operações e expressões regulares

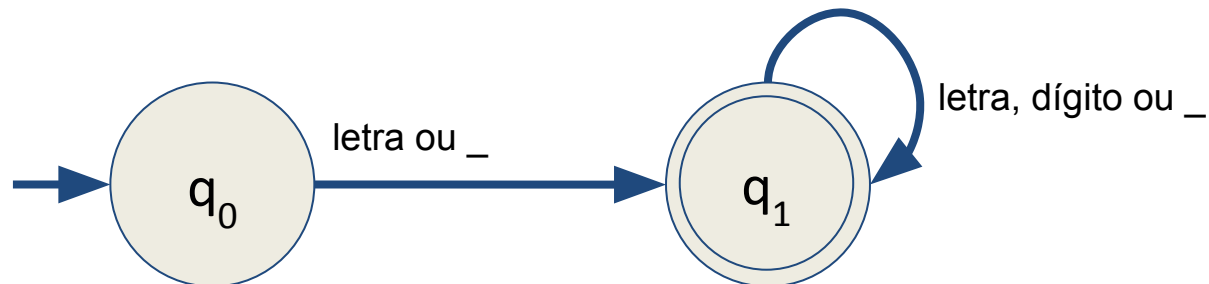
- Tokens podem ser especificados e reconhecidos por expressões regulares.
- Cada classe léxica pode ser associada a uma regra de reconhecimento representada por uma expressão regular.

Classe	Regra (expressão regular)
IDENTIFIER	[A-Za-z_][A-Za-z0-9_]*
INTEGER	[0-9]+
WHITESPACE	[\t\n]+
PLUS	\+
EQUALITY	==

Autômatos

- Autômatos são modelos matemáticos usualmente representados como diagramas de estados capazes de reconhecer linguagens regulares.
- Um autômato finito lê a entrada símbolo a símbolo e muda de estado.

Exemplo: reconhecer um identificador simples (nome de variável) $\rightarrow [A-Za-z_][A-Za-z0-9_]^*$



Autômatos

- Os autômatos podem ser elaborados de duas formas:
- AFD → Autômatos Finitos Determinísticos
- AFND → Autômatos Finitos Não Determinísticos

AFD — determinístico

Para cada estado e símbolo de entrada, há no máximo uma transição.

- Execução direta e previsível
- Muito usado na implementação do scanner

AFND — não determinístico

Pode haver várias transições para o mesmo símbolo e transições ϵ .

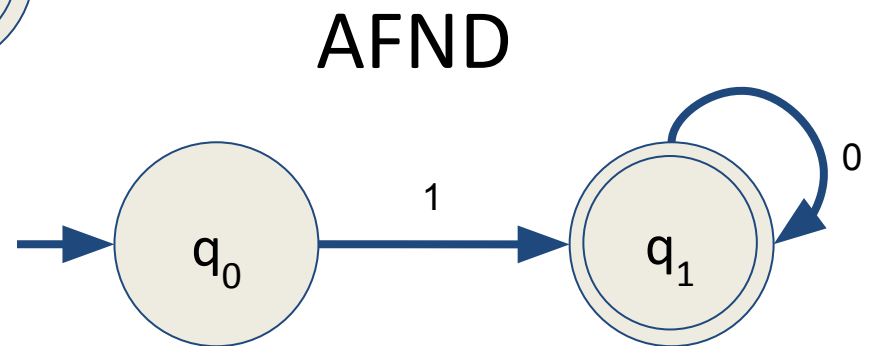
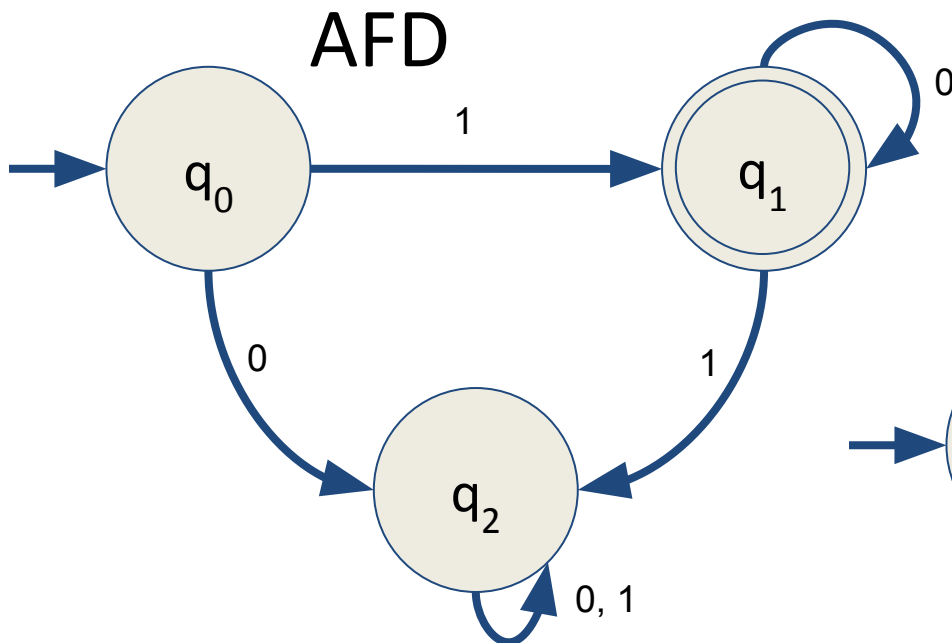
- Mais simples para construir a partir de uma expressão
- Pode ser convertido em AFD

Autômatos

- Exemplo:

$$\Sigma = \{0, 1\}$$

$$L = \{1, 10, 100, 1000, \dots\}$$

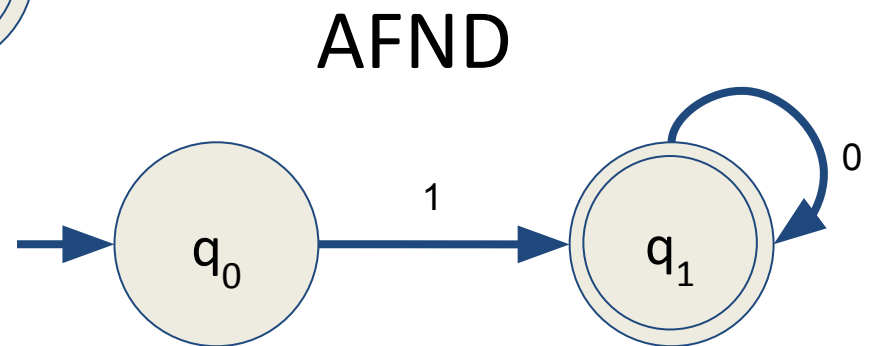
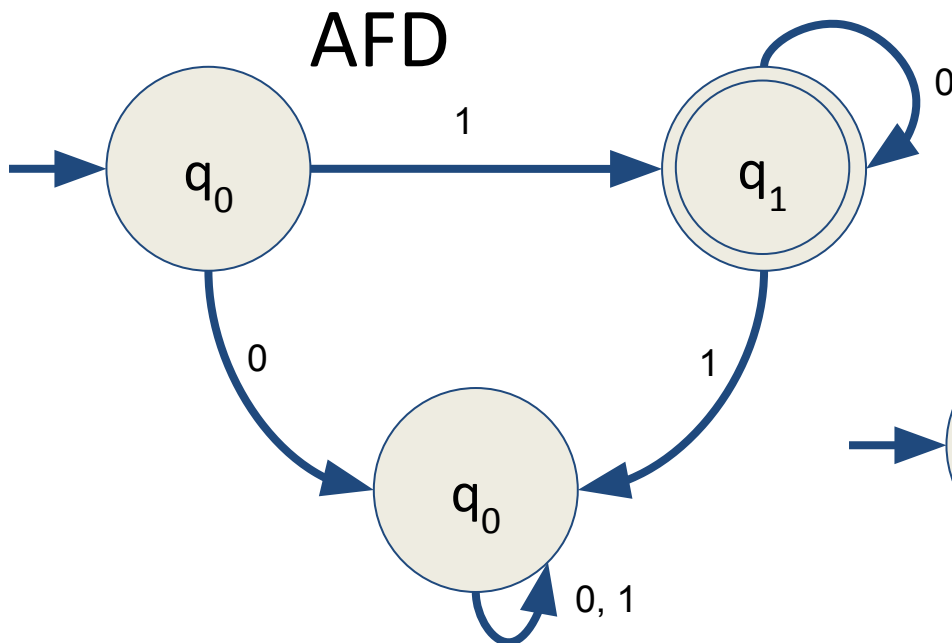


Autômatos

- Exemplo:

$$\Sigma = \{0, 1\}$$

$$L = \{1, 10, 100, 1000, \dots\}$$



Aplicação no projeto MINIC

- O reconhecimento dos lexemas pode ser feito por meio de linguagens regulares.
- Quando mais de uma regra puder ser aplicada, o scanner precisa de uma política clara.

Maior casamento (maximal munch)

Escolha o lexema mais longo possível.

Entrada: a == b

Correto: EQUALITY (==)

Evitar: ASSIGN (=) + ASSIGN (=)

Prioridade entre regras

Se os comprimentos empatarem, use uma ordem definida.

Palavra reservada antes de IDENTIFIER.
Operador de dois caracteres antes do operador simples.

Aplicação no projeto MINIC

- Teste os primeiros recursos do analisador léxico.



Introduza erros léxicos e verifique os diagnósticos.

Aplicação no projeto MINIC

Próximos passos.

01	Definir tokens	Liste palavras reservadas, identificadores, literais, operadores e delimitadores.
02	Escrever padrões	Use expressões regulares e registre ambiguidades.
03	Desenhar autômatos	Escolha um conjunto de tokens e represente seus reconhecedores.
04	Criar testes	Inclua casos válidos, inválidos e de fronteira.